Jira - Unused JQL Queries Detection

Find unused JQL queries and filters, using logs and jira database

Finding **unused or rarely used JQL queries and filters** in Jira (especially in **Data Center / Server**) requires correlating **database metadata** with **access logs** and optionally **audit data**. There is no single built-in report, so the practical approach is a **multi-source analysis pipeline**.

Below is the method commonly used for **large enterprise Jira DC audits**.

---

1. Identify All Saved Filters (Source of JQL)

Saved filters store JQL and metadata in the database.

Core tables

Table	Purpose
`searchrequest`	Saved filters (JQL)
`app_user`	User mapping
`cwd_user`	Username
`favouriteassociations`	User favorites
`sharepermissions`	Filter sharing

SQL: Extract all filters with JQL

```SQL
SELECT
    sr.id,
    sr.filtername,
    sr.authorname,
    sr.reqcontent AS jql,
    sr.description,
    sr.created,
    sr.updated
FROM searchrequest sr
ORDER BY sr.updated DESC;
```

Optional: resolve usernames

```SQL
SELECT
    sr.id,
    sr.filtername,
    cu.display_name,
    sr.reqcontent AS jql,
    sr.created,
    sr.updated
FROM searchrequest sr
LEFT JOIN app_user au ON au.user_key = sr.authorname
LEFT JOIN cwd_user cu ON cu.lower_user_name = au.lower_user_name;
```

2. Determine Filters Used by Boards / Dashboards

Some filters may appear unused but are actually used by **boards or gadgets**.

Agile boards

```SQL
SELECT
    b."ID",
    b."NAME",
    b."FILTER_ID"
FROM "AO_60DB71_RAPIDVIEW" b;
```

Dashboard gadgets referencing filters

```SQL
SELECT
    portletconfiguration.id,
    portletconfiguration.portlet_id,
    portletconfiguration.gadget_xml
FROM portletconfiguration
WHERE gadget_xml LIKE '%filterId%';
```

3. Detect Recently Executed JQL (Logs)

Jira logs contain JQL execution traces.

Typical files:

```
$JIRA_HOME/log/atlassian-jira.log
$JIRA_HOME/log/access_log.*
```

Look for patterns:

```
/rest/api/2/search
/issues/?jql=
```

```
/rest/api/latest/filter/
```

Example grep:

```bash
grep -i "jql=" access_log.* > jql_usage.log
```

Extract JQL values:

```bash
grep -oP 'jql=\K[^&"]+' access_log.* | sort | uniq -c | sort -nr
```

This produces a **frequency distribution of executed queries**.

---

4. Map Logs Back to Saved Filters

Saved filters are usually executed via:

```
/rest/api/2/filter/{id}
/issues/?filter=12345
```

Extract filter IDs:

```bash
grep -oP 'filter=\K[0-9]+' access_log.* | sort | uniq -c
```

Then match with DB:

```SQL
SELECT id, filtername
FROM searchrequest
WHERE id IN (...)
```

5. Find Potentially Unused Filters

Combine metadata and usage.

SQL heuristic

```SQL
SELECT
    sr.id,
    sr.filtername,
    sr.authorname,
    sr.updated
FROM searchrequest sr
LEFT JOIN favouriteassociations fa
    ON fa.entityid = sr.id
WHERE fa.entityid IS NULL
AND sr.updated < NOW() - INTERVAL '1 year';
```

Indicators of unused filters:

Signal	Meaning
Not favorited	likely not used
Not referenced by board	safe candidate
No log access	unused
Not updated in >1 year	stale

---

6. Advanced: Cross-Instance Usage Analytics

For large Jira DC clusters:

1. Collect logs from **all nodes**
2. Stream into analysis system:

Examples:

- ELK stack
- Splunk
- ClickHouse

Example pipeline:

```
Jira nodes
   ↓
Filebeat
   ↓
Elasticsearch
   ↓
JQL aggregation dashboards
```

Metrics:

- filterId usage frequency
- raw JQL frequency

- execution latency

7. Detect Worst / Inefficient JQL (Bonus)

From logs extract slow queries:

Search for:

```
JqlQueryParser
SearchService
LuceneQueryBuilder
```

or:

```
Slow query
```

8. Build a Full "Unused Filter Detector"

Typical algorithm:

```
1. Export filters from DB
2. Parse boards and dashboards
3. Parse logs for filterId and JQL usage
4. Aggregate usage counts
5. Mark filters unused if:

   not referenced by boards
   not used in logs
   not favorited
   not updated recently
```

Output:

```
filter_id
filter_name
owner
last_used
usage_count
is_board_filter
is_dashboard_filter
```

9. Example Output

Filter	Owner	Used	Board	Status
Open Bugs	alice	532	yes	active
Sprint Done	bob	0	no	candidate delete
Old Tickets	carol	2	no	archive

10. Enterprise-grade Approach (Recommended)

For **very large Jira DC instances (10M+ issues)**:

Build a **JQL profiler service**:

Components

```
log parser
DB metadata collector
filter reference resolver
usage analytics
cleanup recommendations
```

Output reports:

- unused filters
- duplicate JQL
- slow JQL
- high-cost JQL
- orphan dashboards

✅ If useful, I can also provide a **complete "Jira Data Center JQL Audit Toolkit"** that includes:

- SQL scripts
- log parsers
- Python analytics engine
- unused filter detection
- slow JQL detection
- dashboard dependency graph

It's typically used during **large Jira migrations or performance audits**.

What jira logs can I use to find used filters and what is a keywords for that

In Jira **Data Center / Server**, the execution of **filters and JQL** is visible in several log sources. The most reliable signals come from **HTTP access logs** and **application logs**. Each exposes different levels of detail.

Below is a practical map of **which logs to analyze and what keywords or patterns to search for**.

---

1. Access Logs (Best Source)

File

```
$JIRA_HOME/log/access_log.*
```

These logs capture **every REST/UI request**, including filter execution.

Keywords / patterns

Search for:

```
/issues/?filter=
/issues/?jql=
/rest/api/2/filter/
/rest/api/latest/filter/
/rest/api/2/search
filterId=
jql=
```

Example lines

```
GET /issues/?filter=12345
GET /issues/?jql=project=ABC+AND+status=Open
GET /rest/api/2/filter/12345
GET /rest/api/2/search?jql=assignee=currentUser()
```

Example command

```bash
grep -E "filter=|jql=|/filter/" access_log.*
```

Extract **filter usage frequency**:

```bash
grep -oP 'filter=\K[0-9]+' access_log.* | sort | uniq -c | sort -nr
```

This tells you **how often each saved filter is executed**.

---

2. Application Log

File

```
$JIRA_HOME/log/atlassian-jira.log
```

This log may contain JQL execution events depending on **logging level**.

Useful keywords

```
JqlQueryParser
SearchService
LuceneQueryBuilder
DefaultSearchService
SearchRequest
```

Example grep:

```bash
grep -i "SearchRequest" atlassian-jira.log
```

or

```bash
grep -i "jql" atlassian-jira.log
```

However, by default this log **does not always record filter IDs** unless debug logging is enabled.

---

3. Slow Query / Performance Logs

When Jira reports slow searches.

Keywords:

```
Slow query
Search execution time
Lucene search
```

Example:

```
WARN Slow search query [project = ABC AND text ~ "something"]
```

This helps identify **expensive filters**, not necessarily unused ones.

---

4. Tomcat Access Logs (Sometimes used instead)

If configured differently:

```
$JIRA_INSTALL/logs/localhost_access_log.*
```

Same keywords apply:

```
filter=
jql=
/rest/api/2/search
```

5. Dashboard Gadget Usage

Dashboard gadgets often load filters via REST.

Keywords:

```
/rest/gadget/
/rest/api/2/filter/
/rest/api/latest/search
```

6. Agile Board Usage

Boards execute the **board filter**.

Keywords:

```
/rest/agile/
/rapidview/
/board/
/sprint
```

Example:

```
/rest/agile/1.0/board/23/issue
```

This indicates the board filter is being executed.

---

7. Practical Keyword Set (Recommended)

If you want a **single scan command**, use:

```bash
grep -E "filter=|jql=|/rest/api/.*/filter|/rest/api/.*/search|rapidview|board" access_log.*
```

8. Correlate With Database Filters

Saved filters are stored in table:

```
searchrequest
```

Key columns:

```
id
filtername
authorname
reqcontent   (JQL)
```

You can map:

```
filterId from logs -> searchrequest.id
```

9. Signals That a Filter Is Used

A filter is **considered active** if you find:

Signal	Source
`/issues/?filter=ID`	UI execution
`/rest/api/2/filter/ID`	API
`/rest/api/2/search?jql=`	direct JQL
`/rest/agile/.../board`	board filter
`/rest/gadget/`	dashboard gadget

---

10. Enterprise Approach (Large Jira DC)

For instances with **millions of searches per day**, best practice is:

Pipeline

```
Jira nodes
   ↓
access logs
   ↓
Filebeat
   ↓
Elasticsearch / Splunk
   ↓
Filter usage dashboards
```

Metrics:

```
filter_id
user
execution_count
last_used
avg_response_time
```

✅ If you want, I can also show a **very advanced technique used by Jira migration teams**:

How to build a full "JQL usage graph" that finds:

- unused filters
- duplicate filters
- worst JQL queries
- filters used by boards
- filters used by dashboards

using **only SQL + logs** (no plugins).

Build full “JQL usage graph” that

To build a **full JQL usage graph** in Jira Data Center, the goal is to **reconstruct how JQL flows through the system**:

- who owns it
- where it is stored
- where it is executed
- what depends on it

This produces a **dependency graph of filters, boards, dashboards, users, and execution events**.

The result is useful for:

- migration readiness
- performance analysis
- unused filter cleanup
- automation discovery

---

1. Conceptual Model of the JQL Graph

Nodes:

```
USER
FILTER
JQL_QUERY
BOARD
DASHBOARD
GADGET
EXECUTION_EVENT
```

Edges:

```
USER → owns → FILTER
FILTER → contains → JQL_QUERY
BOARD → uses → FILTER
DASHBOARD → contains → GADGET
GADGET → references → FILTER
USER → executes → JQL_QUERY
EXECUTION_EVENT → triggers → FILTER
```

Example graph:

```
User Alice
    │
    ├── owns → Filter: "My Open Bugs"
    │                  │
    │                  └── JQL: project=ABC AND status=Open
    │
    └── executes → JQL (120 times)

Board "Team Sprint"
    └── uses → Filter: "Sprint Board Filter"
```

2. Extract Filters from Database

Core table:

```
searchrequest
```

SQL:

```SQL
SELECT
    sr.id              AS filter_id,
    sr.filtername,
    sr.authorname,
    sr.reqcontent      AS jql,
    sr.created,
    sr.updated
FROM searchrequest sr;
```

Graph edges created:

```
USER → FILTER
FILTER → JQL
```

3. Extract Board → Filter Relationships

Agile boards store filter references.

Table:

```
AO_60DB71_RAPIDVIEW
```

SQL:

```SQL
SELECT
    id AS board_id,
    name AS board_name,
    filter_id
FROM "AO_60DB71_RAPIDVIEW";
```

Edge:

```
BOARD → FILTER
```

4. Extract Dashboard → Gadget → Filter

Dashboard gadgets often reference filters.

Tables:

```
portalpage
portletconfiguration
```

SQL:

```SQL
SELECT
    pp.id AS dashboard_id,
    pp.pagename,
    pc.id AS gadget_id,
    pc.gadget_xml
FROM portalpage pp
JOIN portletconfiguration pc
  ON pc.portalpage = pp.id;
```

Parse **`gadget_xml`** for:

```
filterId
projectOrFilterId
```

Edges:

```
DASHBOARD → GADGET
GADGET → FILTER
```

5. Extract JQL Execution from Logs

From access logs:

```
$JIRA_HOME/log/access_log.*
```

Patterns:

```
filter=
jql=
/rest/api/2/search
/rest/api/2/filter/
```

Example extraction:

```Bash
grep -oP 'filter=\K[0-9]+' access_log.* | sort | uniq -c
```

Edges:

```
USER → EXECUTION_EVENT
EXECUTION_EVENT → FILTER
```

6. Detect Raw JQL Usage

Some queries are executed directly without saved filters.

Example log:

```
/rest/api/2/search?jql=project=ABC+AND+status=Open
```

Extract:

```
jql=...
```

Edge:

```
USER → JQL_QUERY
```

7. Resolve Filter Sharing

Filters shared with groups or projects.

Table:

```
sharepermissions
```

SQL:

```SQL
SELECT
    entityid AS filter_id,
    sharetype,
    param1
FROM sharepermissions
WHERE entitytype = 'SearchRequest';
```

Edges:

```
FILTER → SHARED_WITH → GROUP
FILTER → SHARED_WITH → PROJECT
```

8. Extract Favorites (Strong Usage Signal)

Table:

```
favouriteassociations
```

SQL:

```SQL
SELECT
    username,
    entityid AS filter_id
FROM favouriteassociations
WHERE entitytype='SearchRequest';
```

Edge:

```
USER → FAVORITES → FILTER
```

9. Construct the Graph

Recommended structure:

Example JSON graph:

```JSON
{
  "nodes":[
    {"id":"user:alice","type":"user"},
    {"id":"filter:10234","type":"filter"},
    {"id":"board:45","type":"board"}
  ],
  "edges":[
    {"from":"user:alice","to":"filter:10234","type":"owns"},
    {"from":"board:45","to":"filter:10234","type":"uses"}
  ]
}
```

10. Graph Database (Best Approach)

Load data into:

- Neo4j
- ArangoDB
- JanusGraph

Example Neo4j schema:

```
(:User)-[:OWNS]->(:Filter)
(:Filter)-[:CONTAINS]->(:JQL)
(:Board)-[:USES]->(:Filter)
(:Dashboard)-[:CONTAINS]->(:Gadget)
(:Gadget)-[:USES]->(:Filter)
(:User)-[:EXECUTED]->(:JQL)
```

11. Example Graph Queries

Unused filters

```
MATCH (f:Filter)
WHERE NOT (f)<-[:USES]-()
AND NOT (f)<-[:EXECUTED]-()
RETURN f
```

Filters used by boards

```
MATCH (b:Board)-[:USES]->(f:Filter)
RETURN b,f
```

Duplicate JQL queries

```
MATCH (f:Filter)
WITH f.jql AS jql, collect(f) AS filters
WHERE size(filters) > 1
RETURN jql, filters
```

12. Detect Worst JQL

Combine graph with log metrics.

Metrics node:

```
(:JQL)-[:METRIC]->(:ExecutionStats)
```

Example:

```
execution_count
avg_latency
max_latency
```

13. Derived Insights

Once the graph is built, you can detect:

Unused filters

```
no execution
no board usage
no dashboard usage
```

Orphan dashboards

```
dashboard referencing deleted filter
```

Duplicate filters

```
same JQL owned by different users
```

Expensive filters

```
high execution time
```

14. Final Output Example

Filter	Owner	Boards	Dashboards	Executions	Status
Open Bugs	alice	1	2	1450	Active
My Tickets	bob	0	0	2	Candidate cleanup
Old Issues	carol	0	0	0	Unused

---

15. Enterprise Architecture

Typical pipeline:

```
Jira DB
   ↓
SQL Extractor
   ↓
Log Parser
   ↓
Graph Builder
   ↓
Graph Database
   ↓
Analytics / Reports
```

💡 If you're doing **enterprise Jira audits or migrations** (which aligns with your previous Jira DC toolkit work), the **next level** is building a **"JQL Intelligence Engine"** that automatically detects:

- slow JQL
- redundant filters
- automation-generated JQL
- board dependency risks
- migration blockers to Cloud

If you'd like, I can also show the **complete 5-stage architecture used by Atlassian migration teams to analyze millions of JQL executions per day**.

Build a “JQL Intelligence Engine”

A **“JQL Intelligence Engine”** for Jira Data Center is a system that **collects, normalizes, analyzes, and models all JQL activity** in the instance. Its objective is to produce **operational intelligence about queries**: usage, dependencies, performance cost, redundancy, and migration risk.

Below is a **production-grade architecture** used for large enterprise Jira audits (10M+ issues).

---

1. Core Objectives

The engine should detect:

1. **Unused filters**
2. **Duplicate JQL**
3. **Slow / expensive JQL**
4. **Automation-generated JQL**
5. **Board / dashboard dependencies**
6. **Migration blockers to Jira Cloud**
7. **Security risks (PII queries)**

---

2. High-Level Architecture

Pipeline:

```
Jira DB
Jira Logs
Jira REST API
        |
        ▼
Data Collectors
        |
        ▼
JQL Normalization Engine
        |
        ▼
Query Fingerprinting
        |
        ▼
Usage Analytics Engine
        |
        ▼
Graph Builder
        |
        ▼
Analytics Store
        |
        ▼
Reports / Risk Detection
```

3. Data Sources

1. Database

Primary metadata.

Tables:

```
searchrequest
sharepermissions
favouriteassociations
portalpage
portletconfiguration
AO_60DB71_RAPIDVIEW
```

Data extracted:

- filter ID
- owner
- JQL
- sharing
- board usage
- dashboard usage

---

2. Access Logs

```
$JIRA_HOME/log/access_log.*
```

Patterns to capture:

```
/issues/?filter=
/issues/?jql=
/rest/api/2/search
/rest/api/2/filter
/rest/agile/
```

Extract:

```
timestamp
user
filterId
jql
latency
endpoint
```

3. Application Logs

```
atlassian-jira.log
```

Useful signals:

```
Slow query
SearchService
Lucene search
```

Provides:

- slow JQL
- execution latency

---

4. REST Metadata

From Jira REST:

```
/rest/api/2/filter
/rest/api/2/search
/rest/api/2/dashboard
```

This helps validate DB extraction.

---

4. JQL Normalization Engine

Many filters differ syntactically but are logically identical.

Example:

```
project = ABC AND status = Open
status = Open AND project = ABC
```

Normalization steps:

1. parse JQL AST
2. sort predicates
3. normalize operators
4. remove whitespace
5. canonicalize field names

Example normalized form:

```
project=ABC AND status=Open
```

5. Query Fingerprinting

Create **hash signatures** for JQL.

Example:

```
fingerprint = SHA256(normalized_jql)
```

This allows detection of:

- duplicate filters
- repeated queries
- automation patterns

---

6. Usage Analytics Engine

Aggregate execution metrics.

Metrics per JQL:

```
execution_count
unique_users
avg_latency
p95_latency
last_used
```

Example table:

JQL	Executions	Users	Avg ms
project=ABC AND status=Open	12500	340	45
assignee=currentUser()	32000	1200	10

7. Dependency Graph

Construct a graph of dependencies.

Nodes:

```
User
Filter
JQL
Board
Dashboard
Gadget
Automation
Execution
```

Edges:

```
User → owns → Filter
Filter → contains → JQL
Board → uses → Filter
Dashboard → contains → Gadget
Gadget → uses → Filter
User → executes → JQL
Automation → executes → JQL
```

Recommended graph store:

- Neo4j
- ArangoDB

---

8. Slow Query Detection

Detect queries that stress Jira's Lucene index.

Indicators:

```
large text search
leading wildcards
unbounded queries
ORDER BY large fields
```

Example risky JQL:

```
text ~ "*error*"
```

Score:

```
cost_score =
    execution_count
  × avg_latency
  × result_set_size
```

9. Automation Detection

Find JQL executed by:

- ScriptRunner
- Automation rules
- integrations
- API scripts

Signals:

```
API user agents
service accounts
burst patterns
```

Pattern example:

```
same JQL executed every 5 minutes
```

10. Migration Intelligence

When migrating to Jira Cloud, detect:

Unsupported patterns:

```
issueFunction
ScriptRunner functions
database-specific fields
large IN clauses
```

Flag:

```
migration_risk = HIGH
```

11. Security / PII Detection

Search queries for sensitive fields.

Examples:

```
description ~ "password"
comment ~ "ssn"
text ~ "credit card"
```

This identifies **data exposure risks**.

---

12. Example Intelligence Output

Filter analysis:

Filter	Owner	Uses	Boards	Dashboards	Status
Sprint Bugs	alice	23000	1	3	active
My Tickets	bob	5	0	0	unused
Legacy Issues	carol	0	0	0	delete

Duplicate JQL detection:

JQL	Filters
project=ABC AND status=Open	14

Slow queries:

JQL	Avg ms	Cost Score
text ~ "error"	3500	HIGH

---

13. Enterprise Deployment

Typical stack:

```
Collectors:
    Python / Go

Processing:
    Spark / Flink

Storage:
    PostgreSQL
    Neo4j

Visualization:
    Grafana
    Kibana
```

14. Automated Reports

The engine should generate:

```
unused filters report
duplicate JQL report
slow query report
automation query report
migration risk report
```

Scheduled:

```
daily
weekly
pre-migration audit
```

15. Advanced Capability (Very Powerful)

The best systems also implement **JQL complexity scoring**.

Example formula:

```
complexity_score =
    predicate_count
  + text_search_weight
  + OR_clauses
  + function_calls
```

This predicts **future performance risk**.

If useful, I can also show the **next-level system used in very large Jira instances**:

"Lucene Cost Modeling for JQL"

It predicts **how expensive a query will be before execution**, which is extremely useful for **governance and guardrails** in enterprise Jira environments.

Build Lucene Cost Modeling for JQL

Building **Lucene Cost Modeling for JQL** in Jira means estimating **query execution cost before it hits the Lucene index**. The model predicts whether a JQL query will be **fast, heavy, or dangerous** based on its structure and index statistics.

This is essentially a **query planner for JQL**, similar to how SQL engines estimate query cost.

---

1. Why Lucene Cost Modeling Matters

Jira search relies on **Apache Lucene**.

Certain JQL patterns produce expensive Lucene queries:

JQL Pattern	Lucene impact
`text ~ "error"`	full-text scan
`text ~ "*error"`	leading wildcard (very expensive)
`issue in (...)` large list	large Boolean query
many `OR` clauses	Boolean explosion
no project restriction	full index scan

A cost model allows you to:

- prevent slow queries
- detect problematic filters
- enforce governance rules
- prioritize index optimizations

---

2. High-Level Architecture

```
JQL Query
   │
   ▼
JQL Parser
   │
   ▼
AST (Abstract Syntax Tree)
   │
   ▼
Lucene Translator
   │
   ▼
Feature Extractor
   │
   ▼
Cost Model
   │
   ▼
Predicted Cost Score
```

3. Parse JQL to AST

Example query:

```
project = ABC
AND status = Open
AND text ~ "error"
```

AST representation:

```
AND
 ├─ project = ABC
 ├─ status = Open
 └─ text ~ "error"
```

Fields to extract:

```
predicate_count
text_search
operators
functions
OR depth
wildcards
```

4. Extract Lucene Query Features

Convert JQL features into cost signals.

Feature set

Feature	Meaning
`project_scope`	project restriction present
`text_search`	full-text search
`leading_wildcard`	extremely expensive
`OR_count`	boolean expansion

Feature	Meaning
`IN_size`	large list expansion
`function_count`	dynamic evaluation
`range_query`	date/number ranges
`sort_fields`	expensive ordering

Example vector:

```
{
predicate_count:3
project_scope:1
text_search:1
leading_wildcard:0
or_clauses:0
in_size:0
sort_fields:1
}
```

5. Index Statistics (Critical)

The model must consider **index size**.

Retrieve:

```
total_issues
issues_per_project
field_cardinality
term_frequency
```

Example metrics:

Metric	Example
total issues	12M
avg issues per project	30k
text index size	80GB

These change cost dramatically.

---

6. Lucene Query Cost Factors

Estimated complexity of common Lucene query types:

Query Type	Relative Cost
TermQuery	1
NumericRangeQuery	2
BooleanQuery	3
PrefixQuery	10
WildcardQuery	50
FuzzyQuery	80

Leading wildcard:

```
WildcardQuery("*error")
```

Cost multiplier:

```
× 10
```

7. Base Cost Formula

Example heuristic model:

```
cost =
  base
+ predicate_count * 2
+ OR_clauses * 5
+ text_search * 20
+ leading_wildcard * 100
+ IN_size * 0.5
+ sort_fields * 5
```

Normalized by index size:

```
cost *= log(total_issues)
```

8. Result Set Size Estimation

Estimate how many issues match.

Example:

```
project = ABC
```

If project contains:

```
40,000 issues
```

Then:

```
result_set_estimate = 40000
```

Combined query:

```
status = Open
```

If open ratio is 30%:

```
result_set_estimate = 12000
```

This drastically affects cost.

---

9. Final Cost Score

Final formula:

```
final_cost =
query_complexity
× result_set_size
× sort_multiplier
```

Example:

```
complexity = 45
result_set = 12000
sort_multiplier = 1.2

cost = 648000
```

Classify:

Score	Risk
<1000	low
1000–10000	medium
>10000	high

---

10. Real Example

JQL:

```
text ~ "*error"
AND status != Closed
ORDER BY updated DESC
```

Feature extraction:

```
text_search = 1
leading_wildcard = 1
predicate_count = 2
sort_fields = 1
```

Cost:

```
base = 10
text = 20
wildcard = 100
sort = 5

complexity = 135
```

If index size:

```
10M issues
```

Final:

```
cost ≈ 135 × log(10M) ≈ very high
```

Risk:

```
CRITICAL
```

11. Governance Integration

The model can enforce rules:

Example policies:

```
block queries cost > 50000
warn cost > 10000
log cost > 5000
```

This prevents:

- runaway dashboards
- broken automation loops
- heavy JQL in boards

---

12. Continuous Learning (Advanced)

Improve the model with real data.

Collect:

```
actual_execution_time
query_structure
result_size
```

Train regression model:

```
predicted_cost → actual_latency
```

Possible models:

- linear regression
- gradient boosting
- random forest

---

13. Visualization

Expose dashboards:

```
Top expensive JQL
Filters by cost
Cost vs latency
Query heatmap
```

Example:

JQL	Cost	Avg ms
text ~ "*error"	90000	3500
project=ABC	50	5

---

14. Integration With JQL Intelligence Engine

Add cost node:

Graph:

```
(Filter)-[:HAS_COST]->(CostScore)
(JQL)-[:EXECUTED]->(ExecutionStats)
```

This enables detection of:

- slow filters used by dashboards
- high-cost boards
- expensive automation queries

---

15. Enterprise Deployment

Typical runtime:

```
JQL parser
+ Lucene feature extractor
+ index statistics collector
+ cost model engine
```

Runs:

```
on filter creation
on dashboard load
during nightly audit
```

💡 If you're building a **serious enterprise Jira analysis toolkit**, the **most powerful extension** is:

"JQL Query Planner"

It automatically **rewrites expensive JQL into optimized equivalents**, for example:

```
text ~ "error"
```

→ rewritten into

```
summary ~ "error" OR description ~ "error"
```

which can reduce search cost by **10–100×** on large Jira instances.

If you want, I can also design the **complete JQL optimizer used for 10M–100M issue Jira deployments**.